

MMM, Sprawozdanie projekt 6

Ivan Tymoshchenko 206620, Edie Kulik 203578, ACiR 3A

1 Przedstawienie problemu

Celem projektu było opracowanie i zaimplementowanie symulacji (odpowiedzi czasowe) w środowisku Python układu silnika elektrycznego prądu stałego. Sygnałem wejściowym układu jest napięcie (sygnał harmoniczny, prostokątny i trójkątny), które jest podawane na twornik silnika, a sygnałami wyjściowym są prąd płynący w tworniku oraz prędkość obrotowa wału wirnika. Układ składa się z części elektrycznej (obwód twornika) oraz części mechanicznej (wirnik), w części analitycznej zostały wyznaczone wzory dla obu części i połączone ze sobą za pomocą wzorów na moment obrotowy (zależny od prądu) i wsteczną siłę elektromotoryczną (zależną od prędkości wału). Jest to stabilny, co zostało pokazane na końcu części analitycznej, liniowy, stacjonarny układ dynamiczny.

2 Część analityczna



$$T = K_T i \quad e_m = K_e \dot{\theta}$$

Część elektryczna:

$$Ri(t) + U_L + e_m - u(t) = 0$$

$$u(t) - Ri(t) - L\dot{i}(t) - K_e\dot{\theta}(t) = 0$$

Część mechaniczna:

$$J\ddot{\theta}(t) = T - k\theta(t)$$

$$J\ddot{\theta}(t) = K_T i(t) - k\theta(t)$$

Zamiana na dziedzinę stosując transformatę Laplace'a przy zerowych warunkach początkowych.

Następnie wyznaczenie wzoru na $I(s) / \theta(s)$ z jednego wzoru i wstawienie do drugiego, żeby wyznaczyć transmitancję $\frac{s\theta(s)}{U(s)} / \frac{I(s)}{U(s)}$

Transmitancje:

$$U(s) - RI(s) - sLI(s) - sK_e\theta(s) = 0$$

$$\begin{cases} s^2 J \theta(s) = K_T I(s) - k \theta(s) \\ K_T I(s) = s^2 J \theta(s) + k \theta(s) \\ I(s) = \frac{s^2 J \theta(s) + k \theta(s)}{K_T} \end{cases}$$

$$U(s) - \frac{R\theta(s)(s^2 J + k)}{K_T} - \frac{sL\theta(s)(s^2 J + k)}{K_T} - sK_e\theta(s) = 0$$

$$U(s) = \theta(s) \left(\frac{R(s^2 J + k)}{K_T} + \frac{sL(s^2 J + k)}{K_T} + sK_e \right)$$

$$U(s) = \theta(s) \left(\frac{s^2 RJ + RK + s^3 LJ + sLk + sK_e K_T}{K_T} \right)$$

prędkość kątowa:

$$G(s) = \frac{s\theta(s)}{U(s)} = \frac{K_T s}{LJ s^3 + RJ s^2 + (Lk + K_e K_T) s + RK}$$

$$(s^2 J + k) \Theta(s) = K_T I(s)$$

$$\Theta(s) = \frac{K_T I(s)}{s^2 J + k}$$

$$U(s) = RI(s) + sLI(s) + \frac{sK_e K_T I(s)}{s^2 J + k}$$

$$U(s) = I(s) \left(R + sL + \frac{sK_e K_T}{s^2 J + k} \right)$$

$$U(s) = \left(\frac{s^2 J R + kR + s^3 L J + s L K + s K_e K_T}{s^2 J + k} \right) I(s)$$

prąd:

$$G(s) = \frac{I(s)}{U(s)} = \frac{J s^2 + k}{L J s^3 + J R s^2 + (L k + K_e K_T) s + K_R}$$

Powrót do wzorów w dziedzinie czasu i wyznaczenie równań stanu z przekształceń i dodania prędkości kątowej (pochodnej od kąta)

Równania stanu: (i, ω, θ)

$$u(t) - Ri(t) - L\dot{i}(t) - K_e \dot{\theta}(t) = 0 \quad u(t) = Ri(t) + L\dot{i}(t) + K_e \dot{\theta}(t)$$

$$J\ddot{\theta}(t) = K_T i(t) - k\theta(t) \quad L\dot{i}(t) = -Ri(t) - \omega(t) + u(t)$$

$$u(t) = Ri(t) + L\dot{i}(t) + K_e \dot{\theta}(t) \quad J\dot{\omega}(t) = K_T i(t) - k\theta(t)$$

$$J\dot{\omega}(t) = K_T i(t) - k\theta(t) \quad \dot{\theta}(t) = \omega(t)$$

$$\begin{cases} \dot{i}(t) = -\frac{R}{L}i(t) - \frac{K_e}{L}\omega(t) + \frac{1}{L}u(t) \\ \dot{\omega}(t) = \frac{K_T}{J}i(t) - \frac{k}{J}\theta(t) \\ \dot{\theta}(t) = \omega(t) \end{cases} \quad \begin{bmatrix} \dot{i}(t) \\ \dot{\omega}(t) \\ \dot{\theta}(t) \end{bmatrix} = \begin{bmatrix} -\frac{R}{L} & -\frac{K_e}{L} & 0 \\ \frac{K_T}{J} & 0 & -\frac{k}{J} \\ 0 & 1 & 0 \end{bmatrix} \cdot \begin{bmatrix} i(t) \\ \omega(t) \\ \theta(t) \end{bmatrix} + \begin{bmatrix} \frac{1}{L} \\ 0 \\ 0 \end{bmatrix} u(t)$$

Sprawdzenie, czy układ jest stabilny $M(s) = LJs^3 + Js^2 + (Lk + K_e K_T)s + K_R$

$$\begin{matrix} s^3 & L J & L k + K_e K_T \\ s^2 & J R & K_R \\ s & K_e K_T \\ s^0 & K_R \end{matrix} \quad \downarrow \quad \begin{matrix} L > 0, J > 0, K_e > 0, K_T > 0, k > 0, R > 0 \\ \text{Układ jest stabilny} \rightarrow \text{bieguny mają ujemne części rzeczywiste} \end{matrix}$$

3 Opis kodu

Plik `simulation.py` zawiera kluczowe funkcje do symulowania układu.

3.1 Model stanowy

Funkcja `make_state_model()` tworzy macierze modelu stanowego (A, B, C, D) na podstawie parametrów układu (R, L, k itd.).

```
1 def make_state_model(R, L, K_T, K_e, J, k):
2     A = np.array([[ -R/L, -K_e/L, 0],
3                   [K_T/J, 0, -k/J],
4                   [ 0, 1, 0]])
5     B = np.array([[1/L],
6                   [ 0],
7                   [ 0]])
8     C = np.array([[1, 0, 0],
9                   [0, 1, 0],
10                  [0, 0, 1]])
11     D = np.array([[0],
12                  [0],
13                  [0]])
14     return A, B, C, D
```

Macierz A jest macierzą stanu, i opisuje zachowanie swobodne, czyli zależące tylko od stanu układu.

Macierz B opisuje to, jak wejście wpływa na stan układu.

*Szczególne postaci macierzy A i B pochodzą z części analitycznej.

Macierz C jest macierzą obserwacji, czyli wybiera wartości których stanów przechodzą na wyjście. W naszym przypadku jest to macierz jednostkowa, co powoduje że możemy obserwować wszystkie stany układu niezależnie, z wektora y .

Macierz D jest macierzą sprzężenia bezpośredniego i w rzeczywistych układach równa 0, ponieważ inaczej byłaby natychmiast odpowiedź na wejście, co jest fizycznie niemożliwe.

3.2 Symulacja i całkowanie

Symulacja układu polega na obliczeniu pochodnych stanów z macierzy A i stanów, oraz macierzy B i wejścia. Następnie, na podstawie obliczonych pochodnych należy aktualizować stany, całkując pochodne w czasie. Jedną z metod całkowania, która jest bardziej dokładna niż metoda prostokątów, i przy tym nie jest zbyt skomplikowana w implementacji, jest metoda Rungego-Kutty 4 stopnia. Jej realizacja odbywa się przez funkcję `rk4_step()`.

```
1 def rk4_step(x, u, dt, A, B, C, D):
2     y = C @ x + D * u
3     def dxdt(x, u):
4         return A @ x + B * u
5     k1 = dxdt(x, u)
6     k2 = dxdt(x + k1*dt/2, u)
7     k3 = dxdt(x + k2*dt/2, u)
8     k4 = dxdt(x + k3*dt, u)
9     x += dt * (k1 + 2*k2 + 2*k3 + k4) / 6
10    return x, y
```

Z tą metodą, całkowanie z krokiem dt polega na obliczeniu kolejnych wektorów pochodnych: k_1 w czasie t_0 , k_2 i k_3 w czasie $t_0 + \frac{dt}{2}$, oraz k_4 w czasie $t_0 + dt$. Zakładamy, że wejście u pozostaje stałe w czasie jednego kroku.

Na końcu, wektor stanu zmienia się o średnią ważoną tych wektorów pochodnych.

Wyjście y układu jest obliczane na początku funkcji, ponieważ ma ono odbywać się ze stanem przed aktualizacją.

Wzory $dxdt = A @ x + B * u$ oraz $y = C @ x + D * u$ (gdzie $@$ oznacza mnożenie macierzowe) oczywiście pochodzą z postaci równań modelu stanowego. Wejście u jest skalarą, dlatego można użyć mnożenia skalarne zamiast macierzowego.

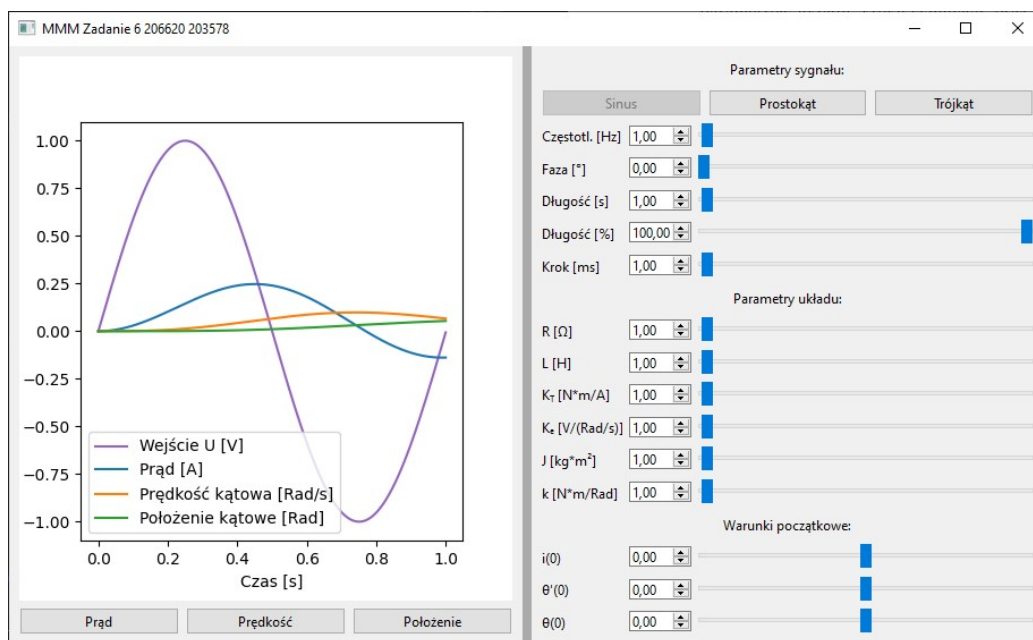
Funkcja zwraca zmodyfikowany wektor stanu x oraz wektor wyjścia y .

3.3 Sygnały

Ponieważ sygnały możemy generować z dowolną rozdzielczością czasową, są one sprzężone z krokiem symulacji, żeby nie mieć trudności z interpolacją.

4 Wnioski

Udało się stworzyć model matematyczny (stanowy) danego układu, oraz zasymulować go odpowiedzi na różne sygnały wejściowe, z wykreśleniem przebiegów każdego ze stanów. Program umożliwia również zmianę parametrów układu, kroku symulacji, oraz warunków początkowych.



Rysunek 1: Okno główne programu

Algorytm całkowania Rungego-Kutty powinien być dość precyzyjny, jednak dla dużych kroków symulacji można zauważyć zniekształcenia. Szczególnie, oscylacje zanikają, a dla niektórych kombinacji parametrów, wartości mogą wybuchnąć do nieskończoności, mimo tego, że teoretycznie układ powinien być stabilny. Wynika to z tego, że najszybsza stała czasowa jest zbyt szybka dla użytej metody całkowania przy tym kroku symulacji.

5 Źródła

<https://www.pythonguis.com/pyside6/>

<https://matplotlib.org>

https://pl.wikipedia.org/wiki/Algorytm_Rungego-Kutty